

REPORT

NLP-Based SMS Spam Detection Using TF-IDF and Multinomial Naive Bayes

Problem Statement

SMS is a common method of communication, but users often receive unwanted messages such as advertisements, fake offers, promotional messages, and fraudulent messages. Manually identifying spam messages can be time-consuming and inefficient.

The objective of this project is to develop an NLP-based SMS spam detection system that automatically classifies SMS messages into **Ham** and **Spam** categories using text preprocessing, TF-IDF feature extraction, and a machine learning algorithm.

Objectives

- To preprocess and clean SMS text using NLP techniques.
- To remove unnecessary words, symbols, numbers, URLs, and stopwords from messages.
- To convert SMS text into numerical features using TF-IDF vectorization.
- To train a Multinomial Naive Bayes classification model.
- To classify SMS messages as Ham or Spam.
- To evaluate the model using accuracy, precision, recall, F1-score, and confusion matrix.
- To test the trained model using new SMS messages.

Dataset

Dataset Name

SMS Spam Collection Dataset

Source

The dataset was provided as a CSV file named spam.csv and used for training and testing the SMS spam detection model.

Dataset Structure

The original dataset contains the following important columns:

- v1 – Message label
- v2 – SMS message text

These columns were renamed as:

- label – Target variable
- message – SMS text

First five records:

	v1	v2	Unnamed: 2	\
0	ham	Go until jurong point, crazy.. Available only ...		NaN
1	ham	Ok lar... Joking wif u oni...		NaN
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...		NaN
3	ham	U dun say so early hor... U c already then say...		NaN
4	ham	Nah I don't think he goes to usf, he lives aro...		NaN

Unnamed: 3 Unnamed: 4

0	NaN	NaN
1	NaN	NaN
2	NaN	NaN
3	NaN	NaN
4	NaN	NaN

Original columns:

```
Index(['v1', 'v2', 'Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'], dtype='object')
```

Dataset shape:

```
(5572, 5)
```

SMS Spam Collection Dataset

Target Variable

The label column was used as the target variable.

- **0 – Ham**
- **1 – Spam**

Data Cleaning

Missing values were checked and removed using `dropna()`.

Duplicate SMS messages were identified and removed using `drop_duplicates()`.

Data Split

The dataset was divided using an **80:20 train-test split** with stratification to maintain the class distribution.

Methodology

Text Preprocessing

The SMS messages were cleaned before training the model using a custom preprocessing function.

The following operations were performed:

- Converted text to lowercase.
- Removed URLs.
- Removed special characters and numbers.
- Tokenized the messages into words.
- Removed English stopwords.
- Applied Porter Stemming.
- Joined the processed words to create the cleaned message.

The cleaned messages were stored in the `clean_message` column.

Original and preprocessed messages:

	message \	clean_message
0	Go until jurong point, crazy.. Available only ...	go jurong point crazi avail bugi n great world...
1	Ok lar... Joking wif u oni...	ok lar joke wif u oni
2	Free entry in 2 a wkly comp to win FA Cup fina...	free entri wkli comp win fa cup final tkt st m...
3	U dun say so early hor... U c already then say...	u dun say earli hor u c already say
4	Nah I don't think he goes to usf, he lives aro...	nah dont think goe usf live around though

Preprocessing

TF-IDF Feature Extraction

TF-IDF was used to convert the cleaned SMS messages into numerical feature vectors suitable for machine learning.

The vectorizer was configured with:

- Maximum features: **5,000**
- N-gram range: **(1, 2)**
- Training data was used to fit the TF-IDF vectorizer.
- Testing data was transformed using the same vectorizer.

TF-IDF helped identify words and word combinations that were important for distinguishing spam messages from normal messages.

Methodology Flow

**Dataset → Data Cleaning → Text Preprocessing → TF-IDF →
Multinomial Naive Bayes → Prediction → Evaluation**

Implementation

The project was implemented in **Python using Google Colab**.

Libraries Used

- Pandas
- NumPy
- Regular Expressions (re)
- NLTK
- Matplotlib
- Seaborn
- Scikit-learn

Machine Learning Model

Multinomial Naive Bayes

Multinomial Naive Bayes was selected as the classification algorithm because it is suitable for text classification problems and works effectively with TF-IDF-based text features.

The model was trained using the TF-IDF features extracted from the SMS messages.

After training, the model was used to predict whether unseen SMS messages were **Ham** or **Spam**.

Results

The performance of the Multinomial Naive Bayes model was evaluated using accuracy, precision, recall, and F1-score.

```
Accuracy : 96.71 %  
Precision: 98.99 %  
Recall   : 74.81 %  
F1-Score : 85.22 %
```

Model Performance

The model achieved the following results:

Metric	Score
Accuracy	96.71%
Precision	98.99%
Recall	74.81%
F1-Score	85.22%

The model achieved an accuracy of **96.71%**, showing that it correctly classified most of the SMS messages in the test dataset.

The precision of **98.99%** indicates that most messages predicted as spam were actually spam.

The recall of **74.81%** shows that some spam messages were still classified as ham.

The F1-score of **85.22%** represents the balance between precision and recall.

Classification Report

A classification report was generated for both Ham and Spam classes using:

- Precision
- Recall

- F1-score
- Support

Confusion Matrix

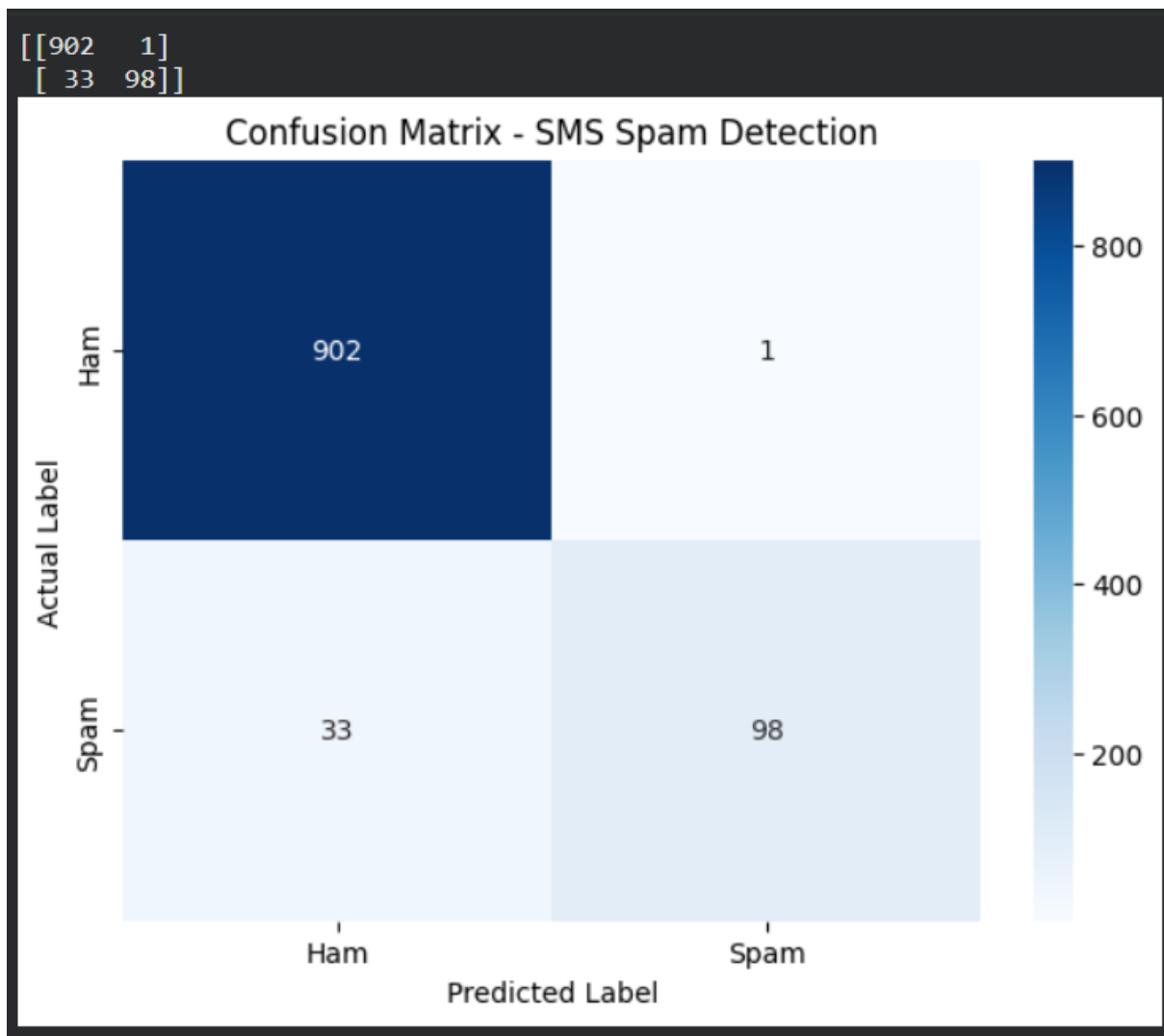
A confusion matrix was generated to visualize the correct and incorrect predictions made by the Multinomial Naive Bayes model.

The confusion matrix contains:

- True Ham
- False Spam
- False Ham
- True Spam

It provides a clear representation of the model's classification performance.

Classification Report



Performance Metrics Graph

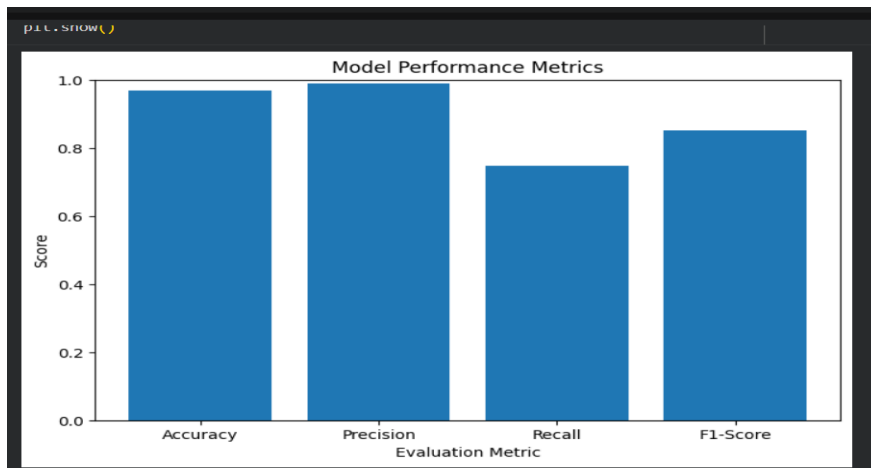
A bar graph was created to visualize the Accuracy, Precision, Recall, and F1-score of the trained model.

Sample Predictions

The trained model was tested using previously unseen SMS messages.

Examples included:

- **"Congratulations! You have won a free prize. Call now!"** → Spam
- **"Hey, are you coming to college tomorrow?"** → Ham
- **"You have been selected for a cash reward. Claim now!"** → Spam
- **"Can you send me the assignment?"** → Ham



Custom SMS Prediction

A custom prediction function was developed to allow users to enter a new SMS message.

The message is first preprocessed and converted into TF-IDF features. The trained Multinomial Naive Bayes model then predicts whether the message is **SPAM** or **HAM**.

```
Message: Congratulations! You have won a free prize. Call now!  
Prediction: SPAM
```

```
Message: Hey, are you coming to college tomorrow?  
Prediction: HAM
```

```
Message: You have been selected for a cash reward. Claim now!  
Prediction: SPAM
```

```
Message: Can you send me the assignment?  
Prediction: HAM
```

Interactive Application

An interactive SMS classification system was also implemented.

The user can enter an SMS message, and the system displays the predicted category. The program continues accepting messages until the user enters exit.

Limitations

The project has the following limitations:

- The model performs only binary classification between Ham and Spam.
- TF-IDF does not fully understand the contextual meaning of a message.
- Sarcasm and complex language may be difficult to classify correctly.
- Some legitimate messages may contain words commonly associated with spam.
- The model is trained specifically on SMS messages and may not perform equally well on other types of text.
- Stemming and removal of stopwords may sometimes remove useful information.

Future Scope

The project can be improved in the following ways:

- Use advanced NLP models such as BERT for better contextual understanding.
- Use larger and more diverse SMS datasets.
- Implement multilingual spam detection.
- Develop a web-based SMS spam detection application.
- Develop a mobile application for real-time spam detection.
- Compare multiple machine learning algorithms such as Logistic Regression, SVM, and Random Forest.
- Improve preprocessing using lemmatization and advanced linguistic

techniques.

- Continuously update the model with new spam messages.

Conclusion

This project developed an NLP-based SMS spam detection system for automatically classifying SMS messages into **Ham** and **Spam** categories.

Text preprocessing was performed using lowercase conversion, URL removal, special character removal, stopword removal, and Porter stemming. TF-IDF vectorization was then used to transform the cleaned SMS messages into numerical features.

A **Multinomial Naive Bayes** machine learning model was trained using these features. The model achieved an **accuracy of 96.71%**, with a precision of **98.99%**, recall of **74.81%**, and F1-score of **85.22%**.

The evaluation included accuracy, precision, recall, F1-score, classification report, confusion matrix, performance graph, and sample predictions.

The results demonstrate that traditional NLP techniques combined with machine learning can provide an effective solution for SMS spam detection. However, the system has limitations in understanding context, sarcasm, and complex language. Advanced NLP models and larger datasets can be used in the future to improve the performance of the system.

References

1. SMS Spam Collection Dataset – spam.csv, used for training and testing the SMS spam detection model.

2. Scikit-learn Documentation – TF-IDF Vectorizer, Multinomial Naive Bayes, train-test split, and evaluation metrics.
3. NLTK Documentation – Stopword removal and Porter Stemmer.
4. Python Documentation – Regular expressions and data processing.